

Explanation

RAG Question-Answering API — one page, covering the four required points.

1. Design parameter: chunk size (800 chars, 120 overlap)

The sample corpus (HR-handbook-style text) is short, self-contained paragraphs, ~100–500 characters each. 800 characters keeps one chunk to roughly one topic (e.g. the whole Leave Policy paragraph, including its three related facts: accrual rate, annual cap, and carry-over rule), so a single retrieved chunk usually contains a complete answer instead of a fragment. Smaller chunks (~200 chars) would split that paragraph across two or three vectors, none of which alone answers the question, forcing retrieval to rank multiple fragments correctly — a harder problem. Larger chunks (2000+) start mixing unrelated topics into one embedding, diluting it. 120-character overlap (15%) exists so a fact sitting near a chunk boundary is still retrievable from either side of it. `chunk_size` and `chunk_overlap` are both `.env`-configurable (app/config.py); a corpus of longer narrative text would want a larger value.

2. Failure actually observed: threshold can't separate relevant from irrelevant queries

The task requires saying “not in the documents” instead of inventing an answer. My first implementation used one cosine-similarity cutoff on the top retrieved chunk. Testing it with the zero-network *offline* embedder (hashing-based bag-of-words, used because this sandbox's egress policy blocks huggingface.co, the source of the recommended *local* sentence-transformers model — confirmed via a 403 policy denial, not a bug) surfaced this:

Query (relevant to the docs)	top sim.	Query (irrelevant)	top sim.
“How many paid leave days...?”	0.171	“Capital of France?”	0.213
“Deadline for expense claims?”	0.275	“How to bake a cake?”	0.133
“On-call compensation?”	0.238	“2011 cricket World Cup winner?”	0.201

The ranges overlap — an irrelevant query (0.213) can score *higher* than a genuinely relevant one (0.171). **Cause:** the hashing embedder is purely lexical (token/token-pair overlap in a fixed-size hashed space); it has no notion that “deadline” and “within 30 days” are related, so incidental word overlap with an unrelated document can outscore real semantic relevance. **Fix applied:** a hard low floor (0.05) catches true non-matches cheaply for every mode; above that, the OpenAI generation path defers to the LLM's own judgment of the retrieved text (reliable — it can read), while only the no-LLM extractive fallback still depends on the raw threshold (documented as unreliable specifically for the offline provider; the local/openai embedders' better-separated score distributions weren't testable in this sandbox for the reason above).

3. Metric tracked: retrieval latency (+ top similarity score, logged alongside)

Every query logs `retrieval_latency_ms`, `generation_latency_ms`, and `top_similarity_score` to `data/metrics.jsonl` (app/utis/metrics.py). Across real test queries against a 2-document corpus, retrieval consistently took **under 1ms** — FAISS IndexFlatIP brute-force search over a handful of vectors is essentially free; almost all end-to-end latency (once an LLM key is set) is the generation call, not retrieval. That tells me this system's latency won't meaningfully change with corpus growth until the vector count is large enough (tens of thousands+) for exact search itself to become the bottleneck — at which point `top_similarity_score`, tracked alongside, is exactly the signal needed to check whether switching to an approximate index (IVF/HNSW) is trading away too much recall for speed.

4. Not finished, and what's next

- **No numerical retrieval eval.** I verified retrieval qualitatively (see §2) but never built a fixed question→expected-chunk labeled set to compute precision/recall. Next: 15–20 labeled Q&A pairs per document, scored automatically on every change to chunking/embedding config.
- **local embedder untested here.** The recommended default (sentence-transformers) couldn't run in this network-restricted sandbox; all testing used the offline fallback. Next: run the same eval set against local and openai on an unrestricted machine and tune `SIMILARITY_THRESHOLD` per provider from real numbers instead of a guess.
- **No document replace/dedup.** Re-uploading a same-named file creates a new `document_id`; old chunks aren't removed from the index. Next: a `DELETE /documents/{id}` endpoint that removes its vectors and metadata.
- **No streaming.** /query blocks until the full LLM response is ready. Next: stream tokens via SSE for better perceived latency on longer answers.